```
import pandas as pd
```

```
df= pd.read_csv('/content/BostonHousing.csv')
```

**Q6. Load the Boston Housing dataset and perform basic data exploration using: **

head() info() describe()

```
df.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64  
 4   nox      506 non-null    float64
 5   rm       506 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64  
 9   tax      506 non-null    int64  
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

```
df.describe()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	5
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.237154	
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.537116	
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.000000	
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000	279.000000	
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.000000	
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000	666.000000	
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.000000	

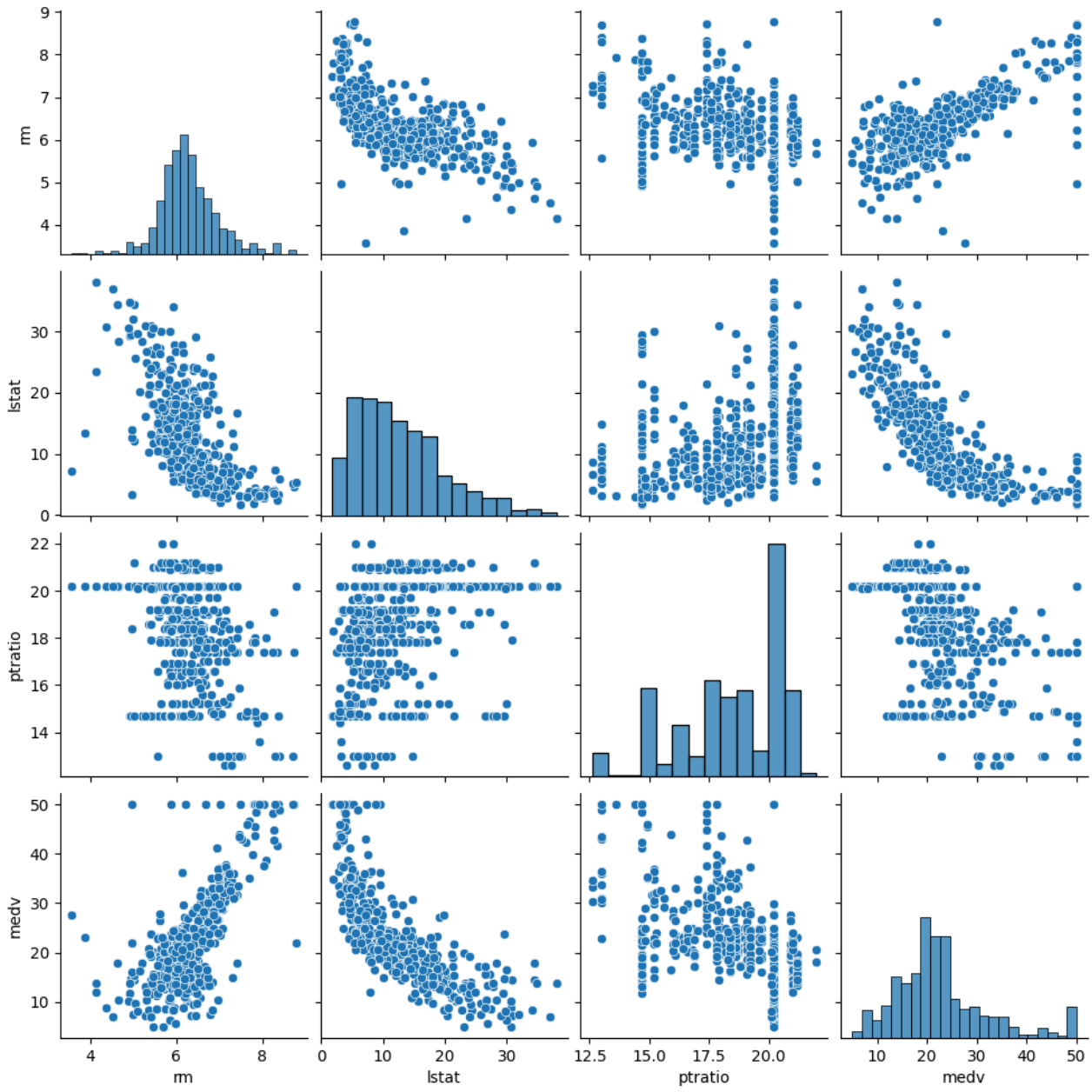
Q7. Perform Exploratory Data Analysis:

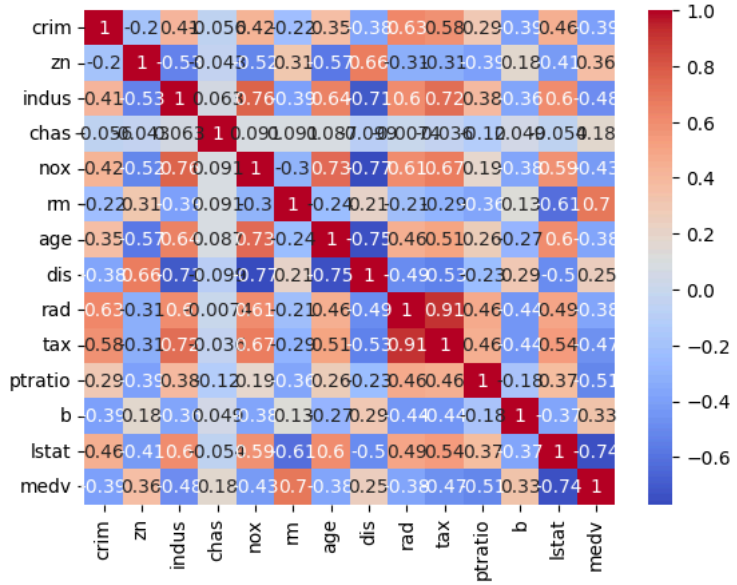
Create pairplot

Create correlation heatmap

Identify which features are highly correlated with house price (medv)

```
import seaborn as sns, matplotlib.pyplot as plt
sns.pairplot(df[['rm','lstat','ptratio','medv']]); plt.show()
sns.heatmap(df.corr(), annot=True, cmap='coolwarm'); plt.show()
```


rm
lstat
ptratio
medv


crim
zn
indus
chas
nox
rm
age
dis
rad
tax
ptratio
b
lstat
medv

```
df
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

Q8. Select independent variables (features) and dependent variable (target).

variable: medv (house price)

```
X = df.drop('medv', axis=1)
y = df['medv']
```

X

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

y

	medv
0	24.0
1	21.6
2	34.7
3	33.4
4	36.2
...	...
501	22.4
502	20.6
503	23.9
504	22.0
505	11.9

506 rows × 1 columns

dtype: float64

Q9. Split the dataset into training and testing sets (80% training, 20% testing)

```
#train model
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test= train_test_split(X,y,test_size=0.2, random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
model.coef_
```

```
array([-1.13055924e-01,  3.01104641e-02,  4.03807204e-02,  2.78443820e+00,
       -1.72026334e+01,  4.43883520e+00, -6.29636221e-03, -1.44786537e+00,
        2.62429736e-01, -1.06467863e-02, -9.15456240e-01,  1.23513347e-02,
       -5.08571424e-01])
```

```
model.intercept_
```

```
np.float64(30.24675099392366)
```

```
y_pred = model.predict(X_test)
```

```
[[y_test,y_pred]]
```

```
[[173    23.6
  274    32.4
  491    13.6
  72     22.8
  452    16.1
         ...
  412    17.9
  436     9.6
  411    17.2
  86     22.5
  75     21.4
  Name: medv, Length: 102, dtype: float64,
  array([28.99672362, 36.02556534, 14.81694405, 25.03197915, 18.76987992,
         23.25442929, 17.66253818, 14.34119   , 23.01320703, 20.63245597,
         24.90850512, 18.63883645, -6.08842184, 21.75834668, 19.23922576,
         26.19319733, 20.64773313,  5.79472718, 40.50033966, 17.61289074,
         27.24909479, 30.06625441, 11.34179277, 24.16077616, 17.86058499,
         15.83609765, 22.78148106, 14.57704449, 22.43626052, 19.19631835,
         22.43383455, 25.21979081, 25.93909562, 17.70162434, 16.76911711,
         16.95125411, 31.23340153, 20.13246729, 23.76579011, 24.6322925 ,
         13.94204955, 32.25576301, 42.67251161, 17.32745046, 27.27618614,
         16.99310991, 14.07009109, 25.90341861, 20.29485982, 29.95339638,
         21.28860173, 34.34451856, 16.04739105, 26.22562412, 39.53939798,
         22.57950697, 18.84531367, 32.72531661, 25.0673037 , 12.88628956,
         22.68221908, 30.48287757, 31.52626806, 15.90148607, 20.22094826,
         16.71089812, 20.52384893, 25.96356264, 30.61607978, 11.59783023,
         20.51232627, 27.48111878, 11.01962332, 15.68096344, 23.79316251,
          6.19929359, 21.6039073 , 41.41377225, 18.76548695,  8.87931901,
         20.83076916, 13.25620627, 20.73963699,  9.36482222, 23.22444271,
         31.9155003 , 19.10228271, 25.51579303, 29.04256769, 20.14358566,
         25.5859787 ,  5.70159447, 20.09474756, 14.95069156, 12.50395648,
         20.72635294, 24.73957161, -0.164237  , 13.68486682, 16.18359697,
         22.27621999, 24.47902364])]]
```

Q10. Train a Linear Regression model and perform the following:

Make predictions

Calculate MAE

Calculate MSE

Calculate RMSE

Calculate R^2 Score

Calculate Adjusted R^2

Write the Linear Regression equation

Interpret the model coefficients

```
from sklearn.linear_model import  LinearRegression
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
import numpy as np


model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
n, p = X_test.shape[0], X_test.shape[1]
adj_r2 = 1 - (1-r2)*(n-1)/(n-p-1)
```

```
mae, mse, rmse, r2, adj_r2
```

```
(3.189091965887837,
 24.291119474973478,
 np.float64(4.928602182665332),
 0.6687594935356326,
 0.619826236898851)
```

```
Start coding or generate with AI.
```